There is no spoon

Load-testing the real world

Henrik Rexed · IsItObservable · PAC 2026

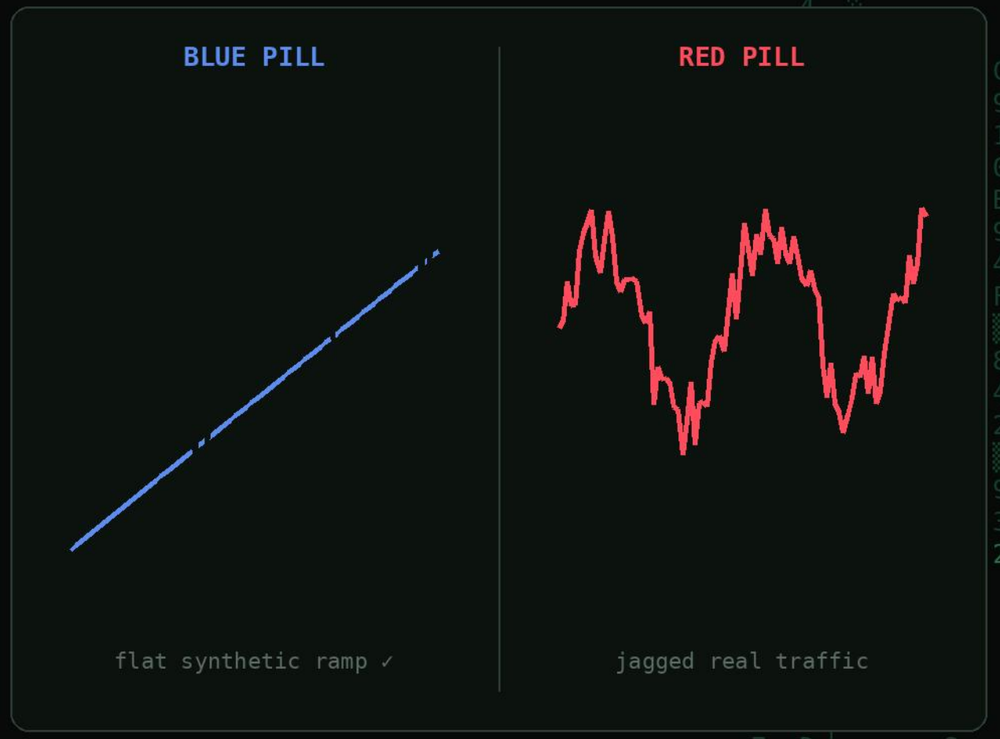


take the red pill — stop guessing, load the real world

SLIDE 2 · THE CHOICE EVERY ENGINEER MAKES

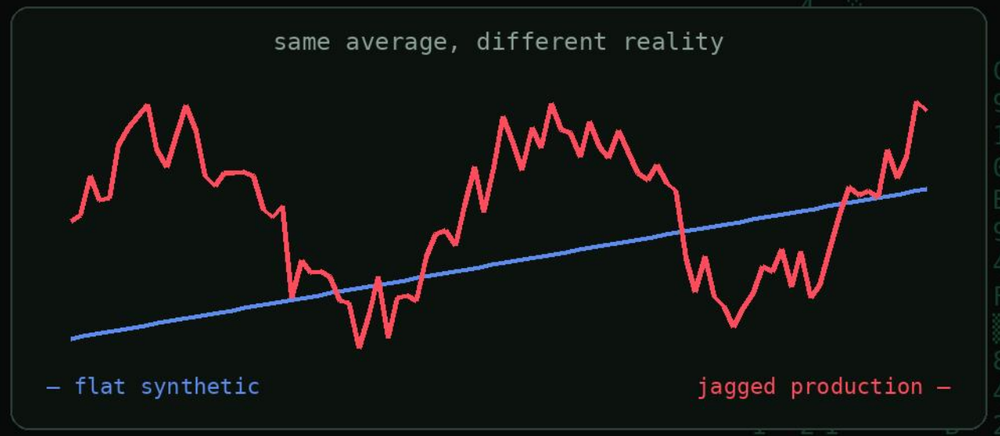
The blue pill load test

- Blue pill: run the test you always run. Dashboard green. Believe what you want.
- Red pill: look at what production actually does.
- “…how deep the rabbit hole goes.”
- **This talk is the red pill.**



A load test must reflect real life

- Volume — “10× peak”? peak of what?
- Mix — 3 endpoints vs 40 in prod
- Pacing — thread-group defaults
- Payloads — 3 hardcoded blobs
- **Shape mismatch = the test tests your imagination**



Volume	Mix	Pacing	Payloads
source: ???	source: ???	source: ???	source: ???

DISCUSSION #1

Where do your numbers come from?

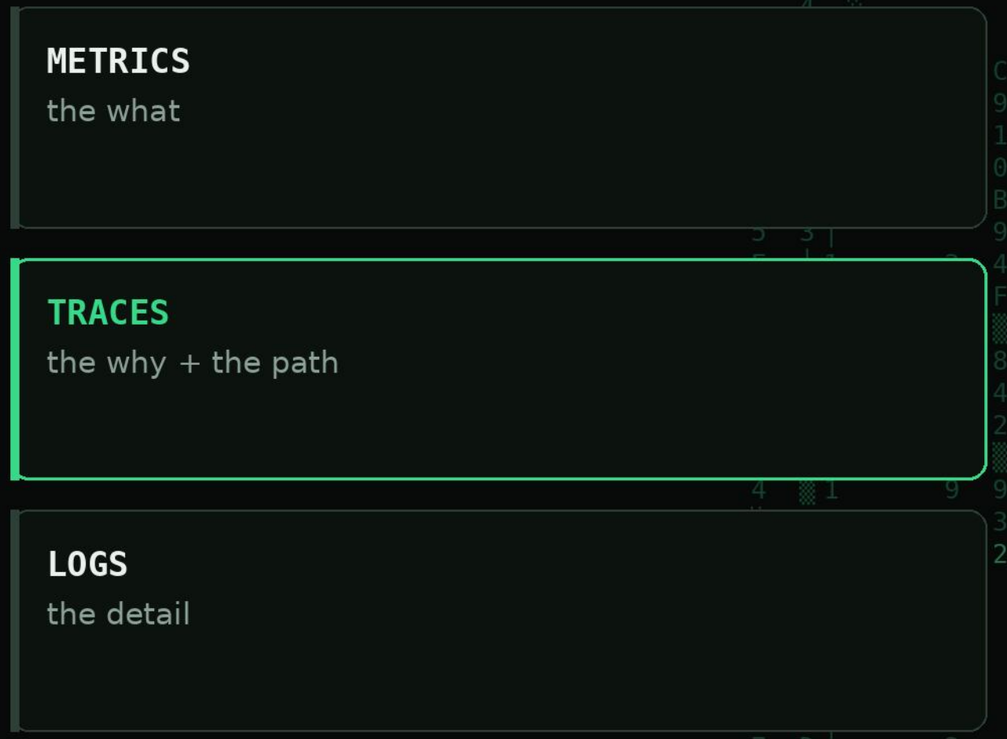
Who chose your RPS — and what was it based on?



> _

The evolution with OTel

- Metrics — the what
- Traces — the why + the path
- Logs — the detail
- For load tests, traces = journeys + fan-out + arrival curve
- **But the red pill shows reality through a sample**



The sampling trap

- Déjà vu = a glitch in the Matrix
- Naive: count × 1/rate — only valid for head sampling
- Head: adjusted count = 1 / probability (OTel spec)
- Tail: reweighting thrown away — you cannot extrapolate counts
- Fix: compare rates, normalize each dataset's ratio

HEAD SAMPLING

decision up front · probability propagated

✓ **clean reweight — counts recoverable**

TAIL SAMPLING

decision after the trace · outcome-biased

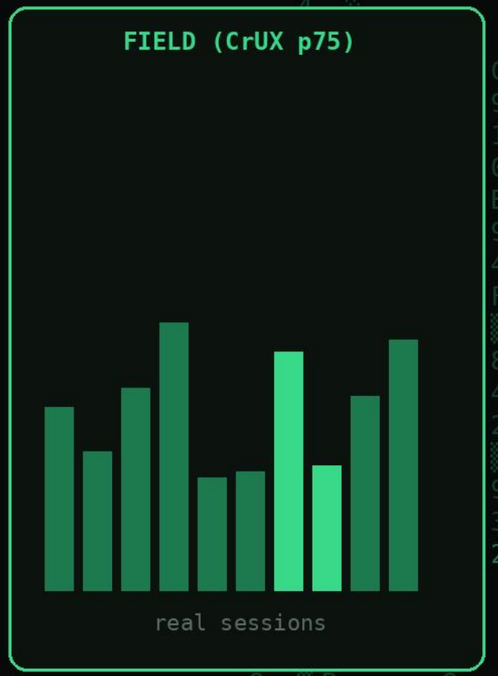
✗ **cannot recover counts/rates**

samplingFactor recorded in shape-diff.json

State of RUM

- RUM = real journeys, real arrival curve, real device/geo/network
- Google ranks on field CrUX @ 75th pct — not Lighthouse (lab)
- OTel browser SDK + session semconv = Development, not stable
- Vendor OTel-RUM = technical preview
- **Production RUM truth today = your backend's user.sessions**



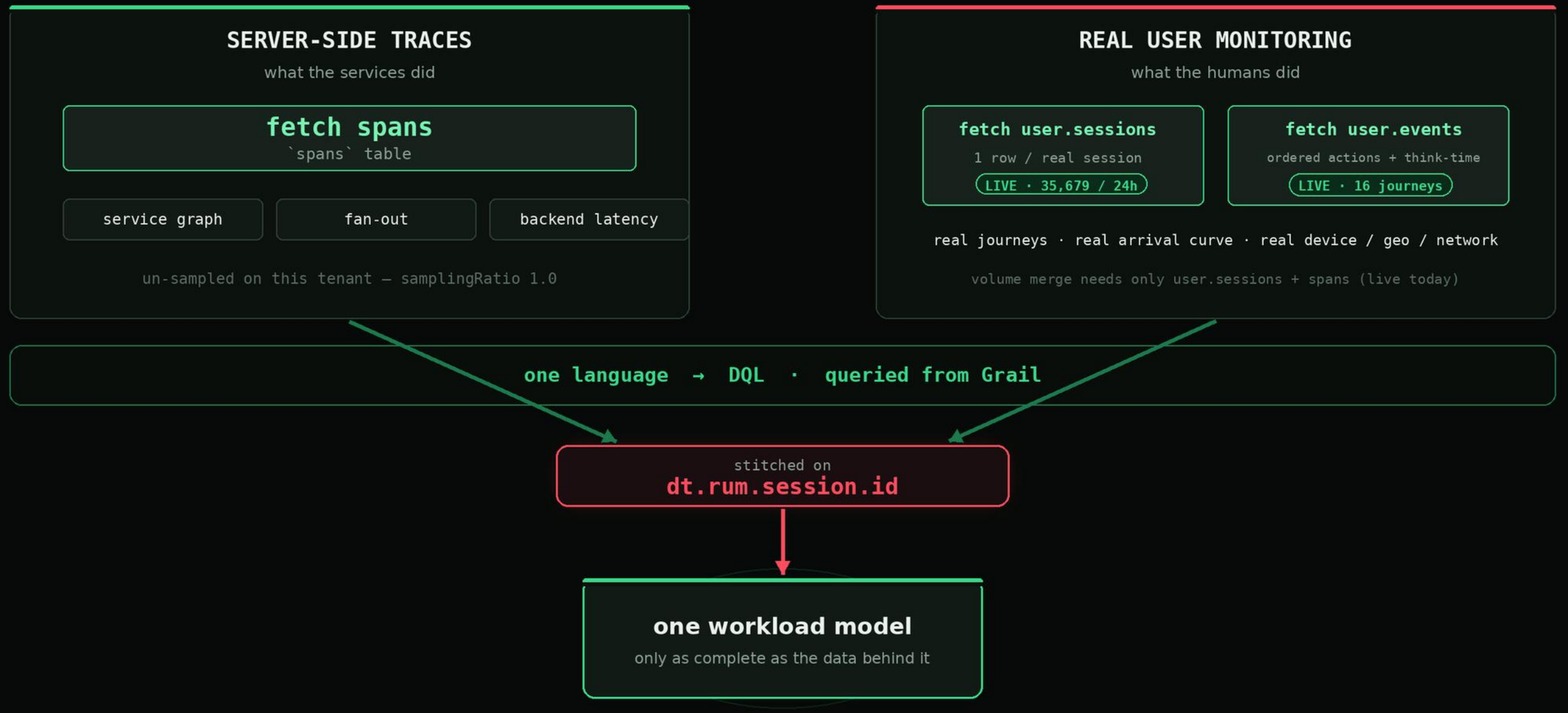


SPANS + RUM · ONE PROJECTION

Where the truth is stored

"Residual self image — the mental projection of your digital self."

Morpheus



spans = what services did · RUM = what humans did · each is blind where the other sees

DQL: querying both worlds

dtctl · DQL

```
# service-side shape
> fetch spans | summarize spans=count(), by:{service.name}

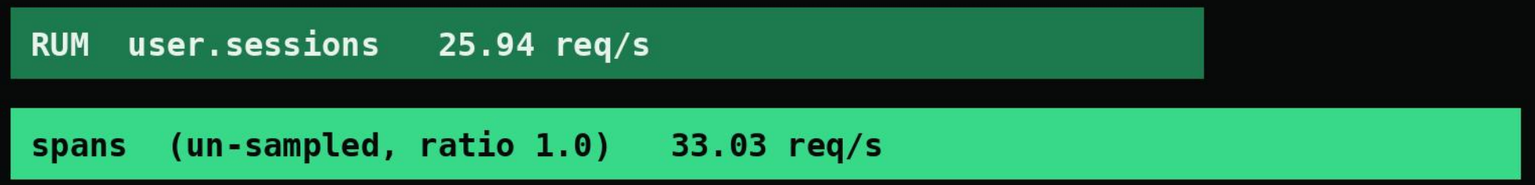
# the real session population  —  live, accessible now
> fetch user.sessions | summarize sessions=count()
    35,679 sessions / 24h · 5 apps (web + mobile)        ● LIVE

# the ordered journey + exact think-times
> fetch user.events | filter dt.rum.session.id == <id> | sort timestamp asc
    scope granted (ISI-5719) → returns rows                ● LIVE
    18,773 sessions → 16 ordered journey templates
```

volume merge + the ×1.27 factor need only user.sessions + spans — user.events is the bonus (degrade-safe)

DISCUSSION #2

Do you trust a single source to size a load test?



×1.27

coverage gap — not sampling

Size off one source → under-load by 21%, call it a pass.

Reconcile sources, record the factor. Same cure as sampling — different disease.

> _

Let's use a skill

- A skill = versioned, deterministic capability
- Reads telemetry → proposes a workload model
- Inside a scope contract: read-only inputs, hard caps, human gate
- **The agent proposes; it does not decide**



ADAPTER → ENGINE → RENDERERS

The engine: telemetry in, model out

"This is the Construct. It's our loading program. We can load anything..."

Morpheus

INPUT ADAPTER

backend-specific

raw telemetry → one neutral schema

- **DQL (this build)**
- Tempo
- Jaeger
- Envoy logs

neutral telemetry →

ENGINE

backend-agnostic · deterministic · offline-testable

- front-end discovery
- scenario clustering → ~80% coverage
- per-scenario load — Little's Law
- ramp shaping
- dataset sizing

same telemetry → same model, every time

→ workload-model.json

RENDERERS

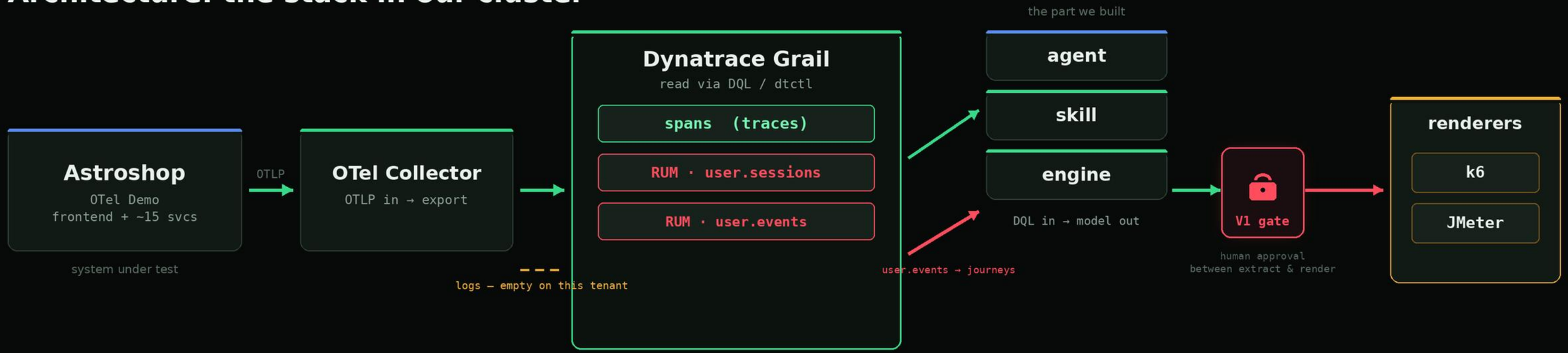
model → runnable world

- k6 ✓
- JMeter ✓
- Locust ✓
- Gatling ✓

nothing about Dynatrace or k6 lives inside the engine — that boundary is the point

THE REAL WIRED STACK — NOT SLIDEWARE

Architecture: the stack in our cluster



read-only · extraction-only · non-production · tenant-isolated

repo
github.com/henrikrexed/AI-Performance-Testing

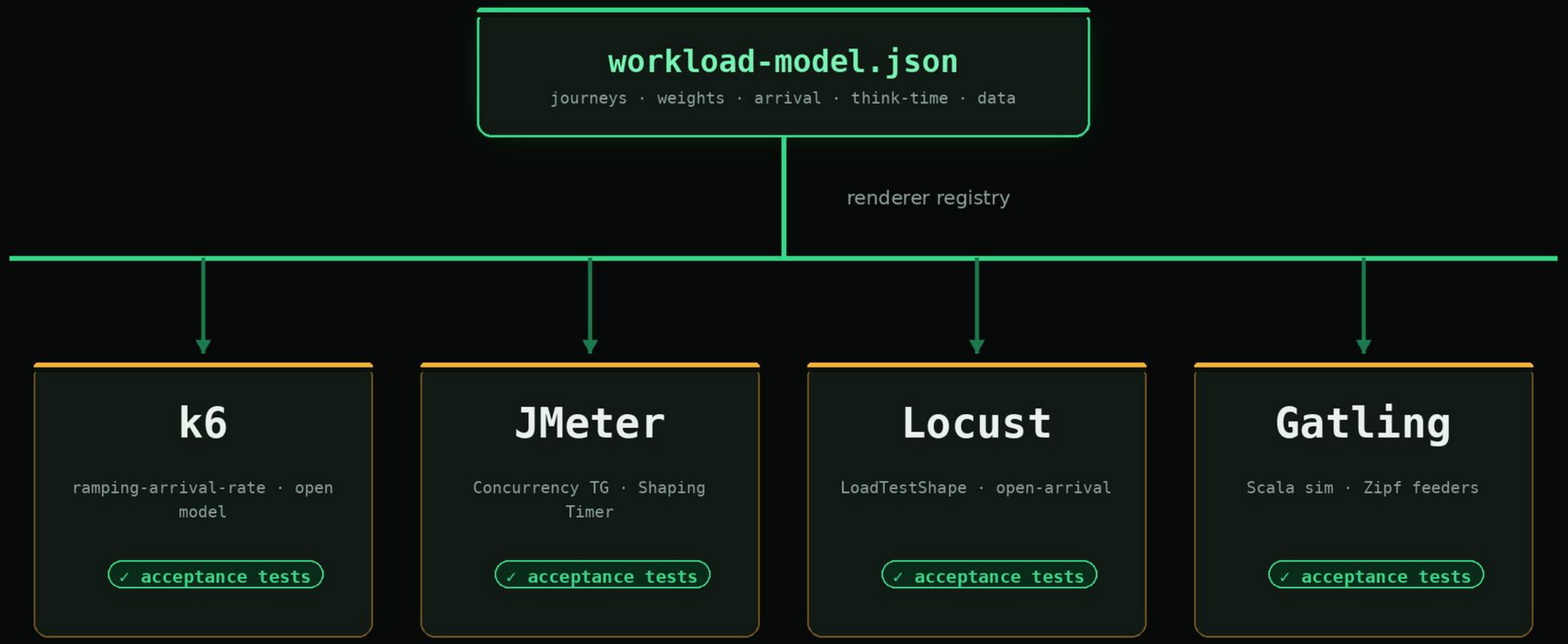
scan → live public mirror (ISI-5659)

ONE MODEL · MANY WORLDS

The renderer registry

"Guns. Lots of guns."

Neo



the model is the asset · the tool is a render target · all four pass acceptance tests today

Load the program — LIVE

● LIVE — DQL vs Astroshop

run-demo-live.sh

```
$ ./run-demo-live.sh

[1/4] fetch spans            → service shape (live DQL)
      root traces ≈ 321k · browser 99% · buyer 1%
[1/4] fetch user.sessions    → real journeys (live)
      samplingRatio 1.0   (this tenant is un-sampled)

      G0 gate … 12 / 12  ✓    ← re-derived live, not constants

      no pre-baked model — materialized from what it sees now
```

“I know kung fu.”

fallback: run-demo.sh (rehearsed recorded run — narrate identically, say once it's the recording)

user.events returns rows — then the Oracle

● LIVE — DQL vs Astroshop

run-demo-live.sh

```
fetch user.events …
  403 not authorized          (until this week — storage scope)
  ↳ scope granted (ISI-5719)  → returns rows  ← the wall came down
  18,773 sessions → 16 ordered journey templates + think-times

note: model sized from spans + user.sessions
      user.events = bonus · degrades cleanly if ever gone

[gate] approve model revision <hash>?  (human)  ▸ y
       the engine will not synthesize a request until I approve
```

Trust the trace, not the demo.

fallback: run-demo.sh (rehearsed recorded run — narrate identically, say once it's the recording)

Render k6 · JMeter

● LIVE — DQL vs Astroshop

run-demo-live.sh

```
[2/4] model → k6
      ramping-arrival-rate · stages scaled off weights
      synthetic=true in baggage on every request
      V2a checks … ✓

[3/4] model → JMeter
      Concurrency Thread Group · Throughput Shaping Timer
      CSV data sets (cache-busting)
      J1–J7 static checks … ✓
```

“Show me.” One approved model → two guns, zero hand-authoring.

fallback: run-demo.sh (rehearsed recorded run — narrate identically, say once it's the recording)

Guns. Lots of guns.

● LIVE — DQL vs Astroshop

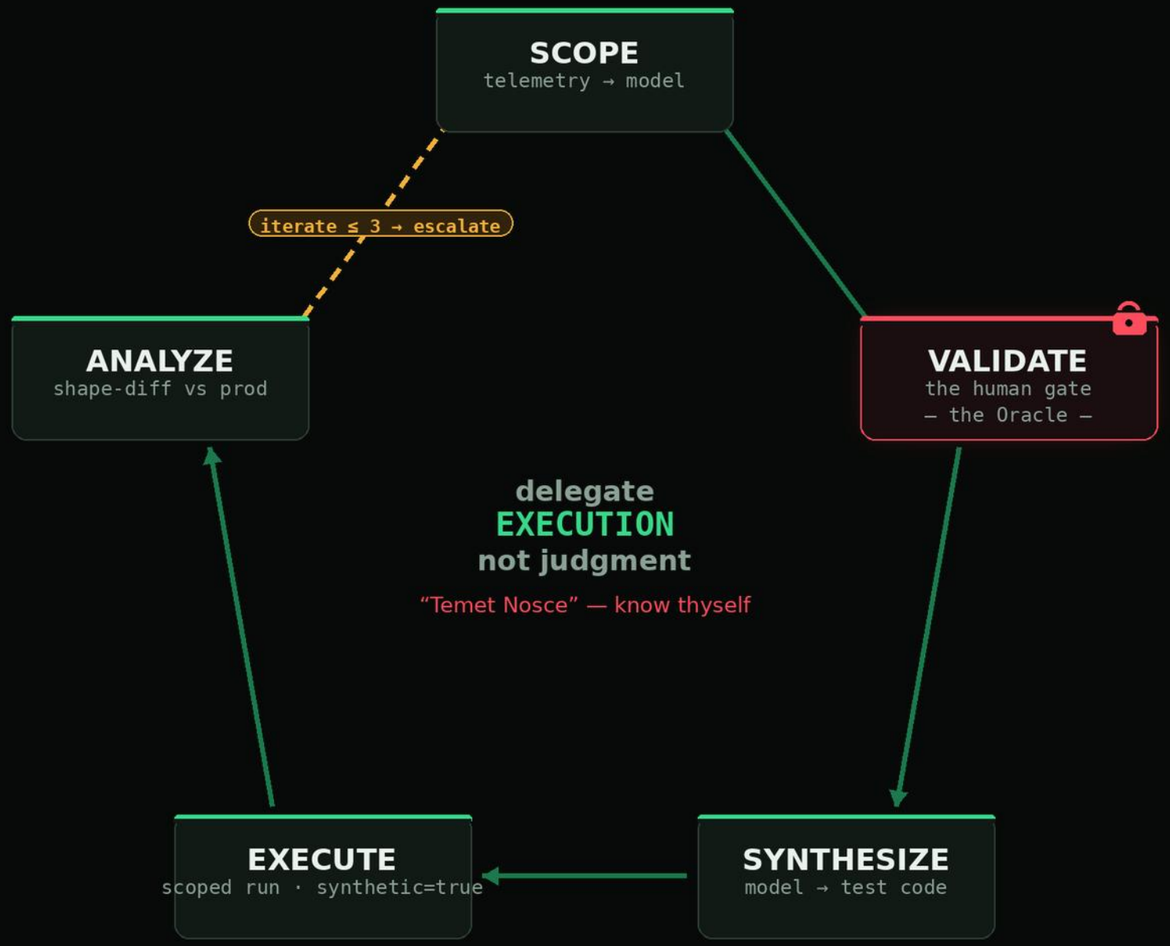
property	k6	JMeter	match
journeys	●	●	✓
weights	●	●	✓
arrival envelope	●	●	✓
synthetic baggage	●	●	✓

The model is the program; the renderer is just the compiler back-end.

fallback: run-demo.sh (recorded) — if time is short, narrate steps 1, 2, 4 and drop JMeter detail

SCOPE · VALIDATE · SYNTHESIZE · EXECUTE · ANALYZE

The loop



VALIDATE is hash-bound to the model revision · change a field → the gate re-opens · no auto-approval, ever

SLIDE 20 · THE CAVEATS AN EXPERT ROOM DESERVES

Honest limits: glitches in the Matrix

R5

Circular ground truth

Extract + validate from the same telemetry.

Cross-check ≥1 run against infra metrics.

R2

Mean ≠ peak

We model mean arrival today.

A peak-envelope profile is on the backlog.

RUM

Ordered journeys

storage:user.events:read now granted (ISI-5719).

Folding them into the rendered model is next.

There is no spoon — don't tune the test green; bend it to reality.

DISCUSSION #3 + CLOSE

What would you red-pill first?

One service you already observe — and the one number you can't justify.

`telemetry proposes → you validate → render → run scoped → diff`

Stop guessing. Load the real world.

github.com/henrikrexed/
AI-Performance-Testing

It's all open — methodology · engine · renderers · this demo